



Universidad Autónoma de Baja California

Programación Orientada a Objetos

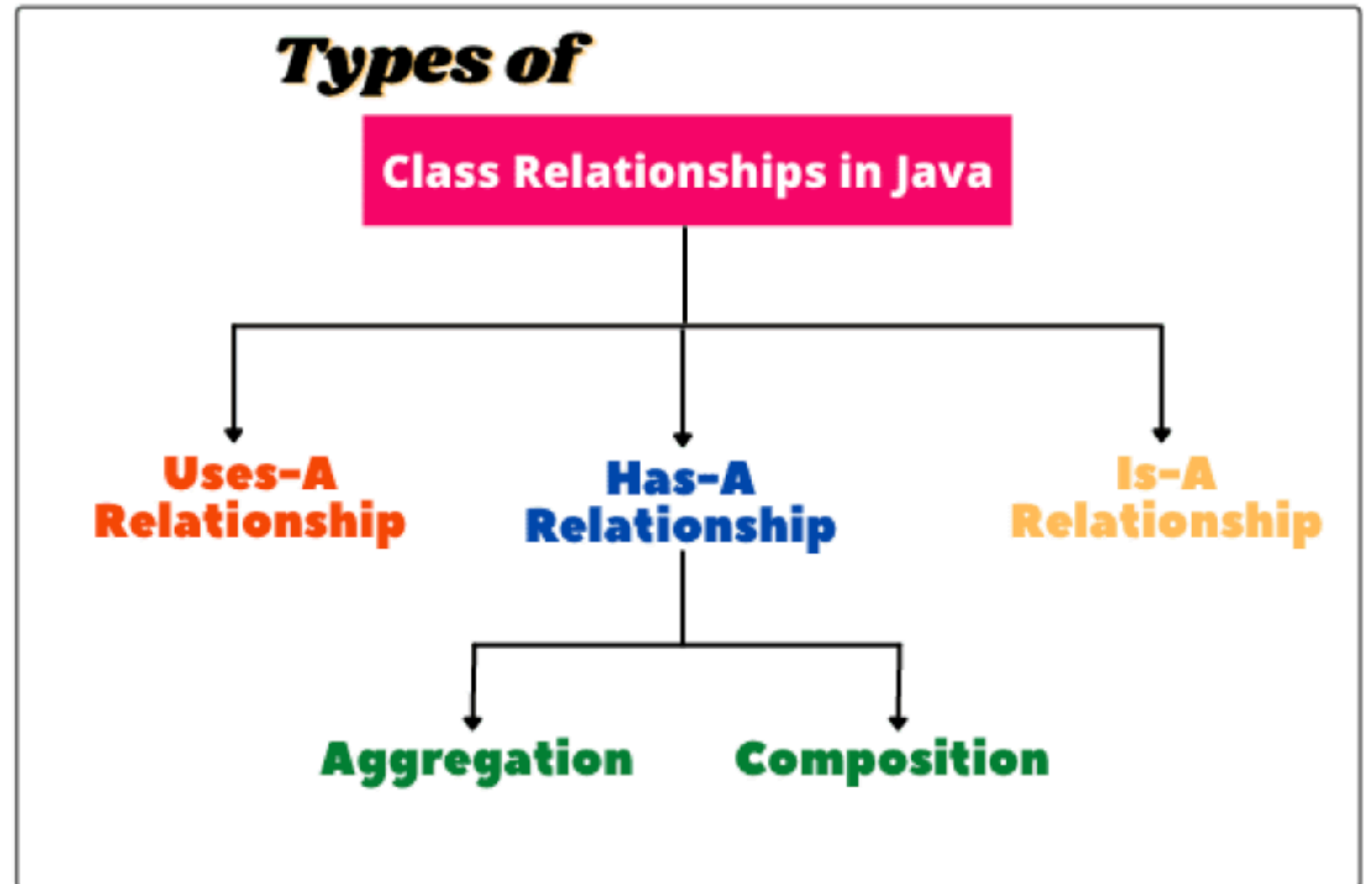
Programación Orientada a Objetos

Relaciones entre clases

Profesor: MC. Itzel Barriba Cázares

Relaciones entre clases

- ▶ En Java, las clases pueden relacionarse entre sí para modelar situaciones del mundo real.
- ▶ Las relaciones más comunes son:
 - ▶ Dependencia (usa un)
 - ▶ Asociación
 - ▶ Composición/Agregación



Asociación

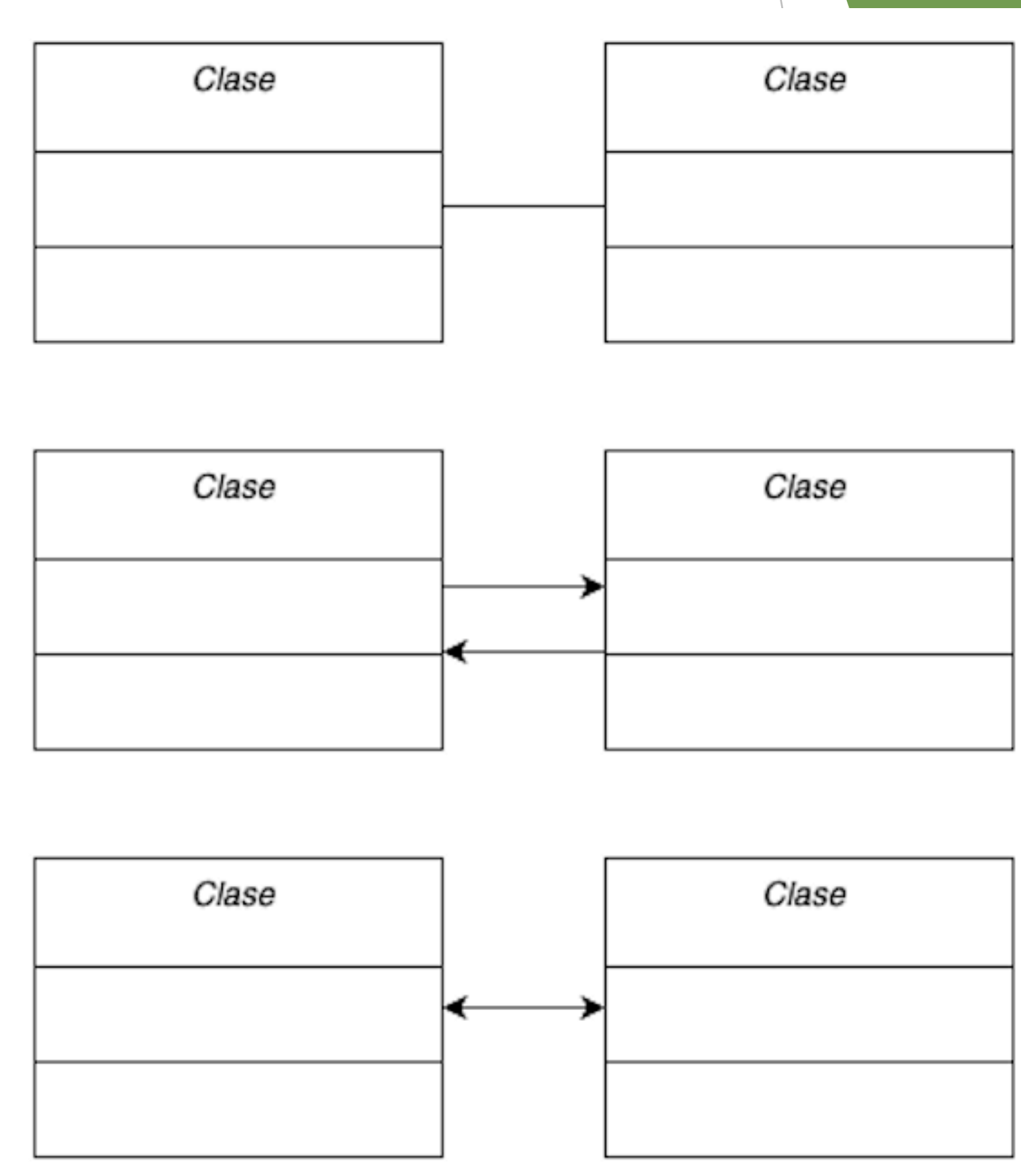
- ▶ La **asociación** es una **relación entre clases independientes**, donde:
 - ▶ Los objetos pueden comunicarse entre sí.
 - ▶ Ninguna clase depende completamente de la otra.
- ▶ Es la relación más flexible y débil.

Asociación

- ▶ La **asociación** se implementa mediante atributos (referencias a objetos)
- ▶ Las clases pueden existir **independientemente**
- ▶ Puede ser:
 - ▶ Unidireccional
 - ▶ Bidireccional

Representación en UML

- ▶ Se representan con **una línea sólida entre clases**
- ▶ Puede incluir flechas (dirección)
- ▶ Puede indicar multiplicidad



Relación de Asociación

Persona



Celular

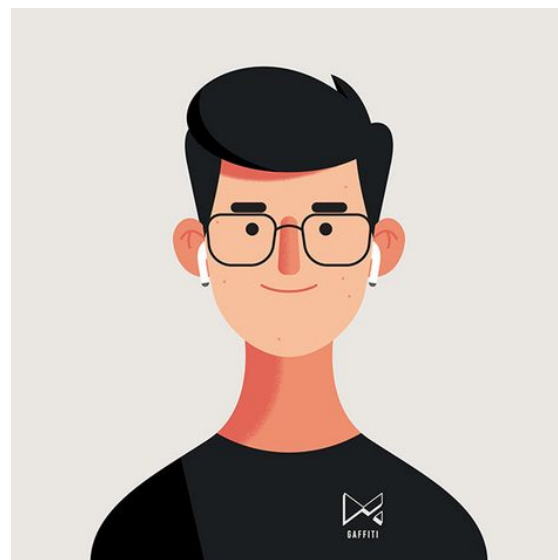


- ▶ Una persona tiene un telefono, pero
 - ▶ La persona puede existir sin teléfono
 - ▶ El teléfono puede existir sin la persona

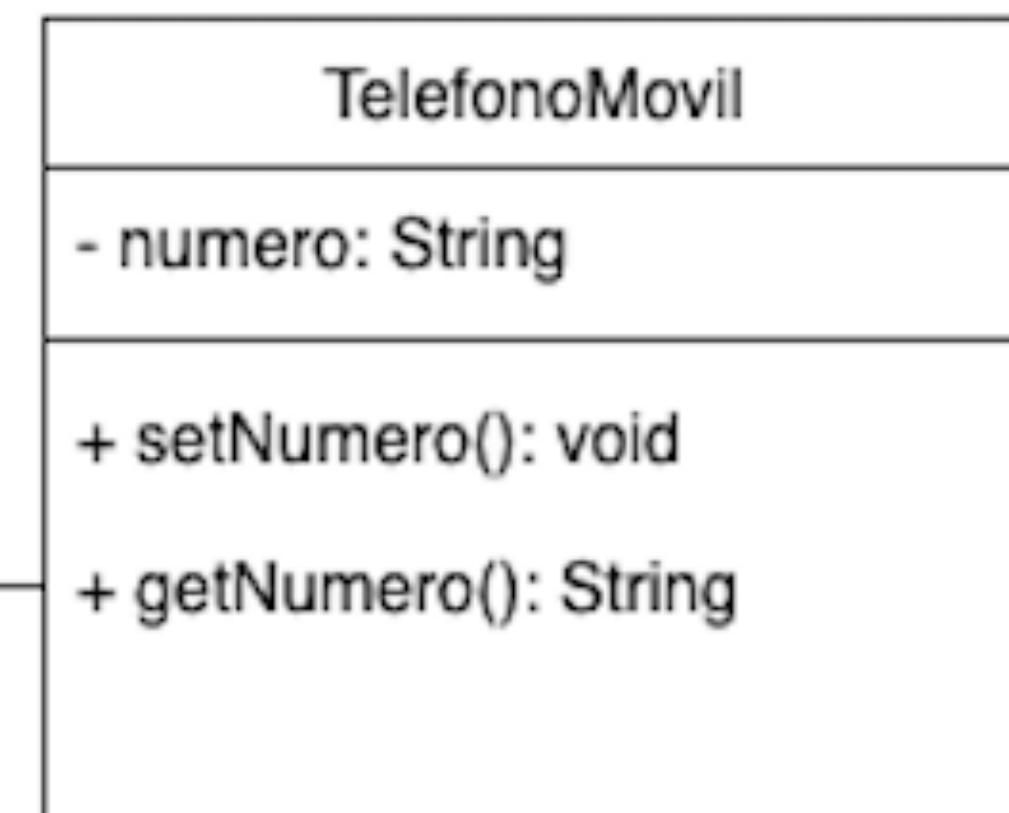
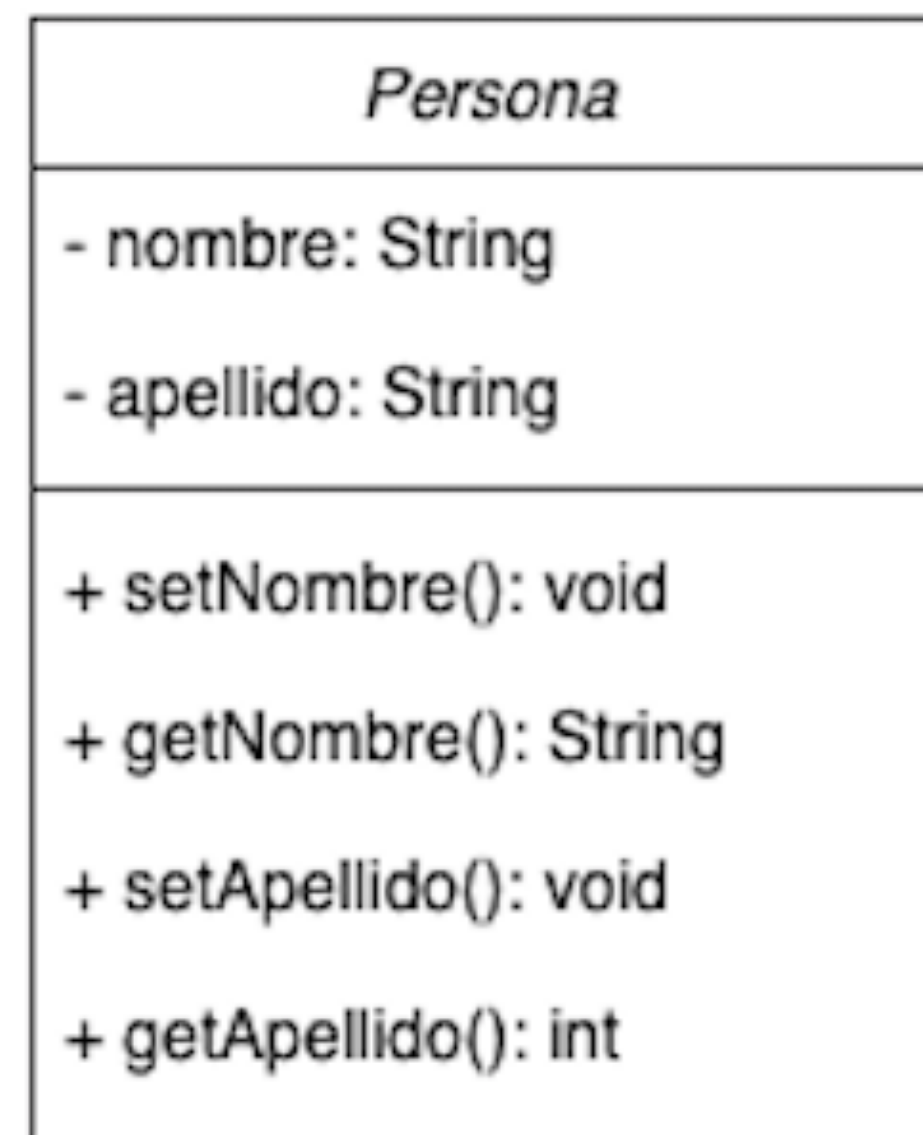
Relación de Asociación

- ▶ Persona y teléfono móvil están asociados a través de sus objetos.

Persona

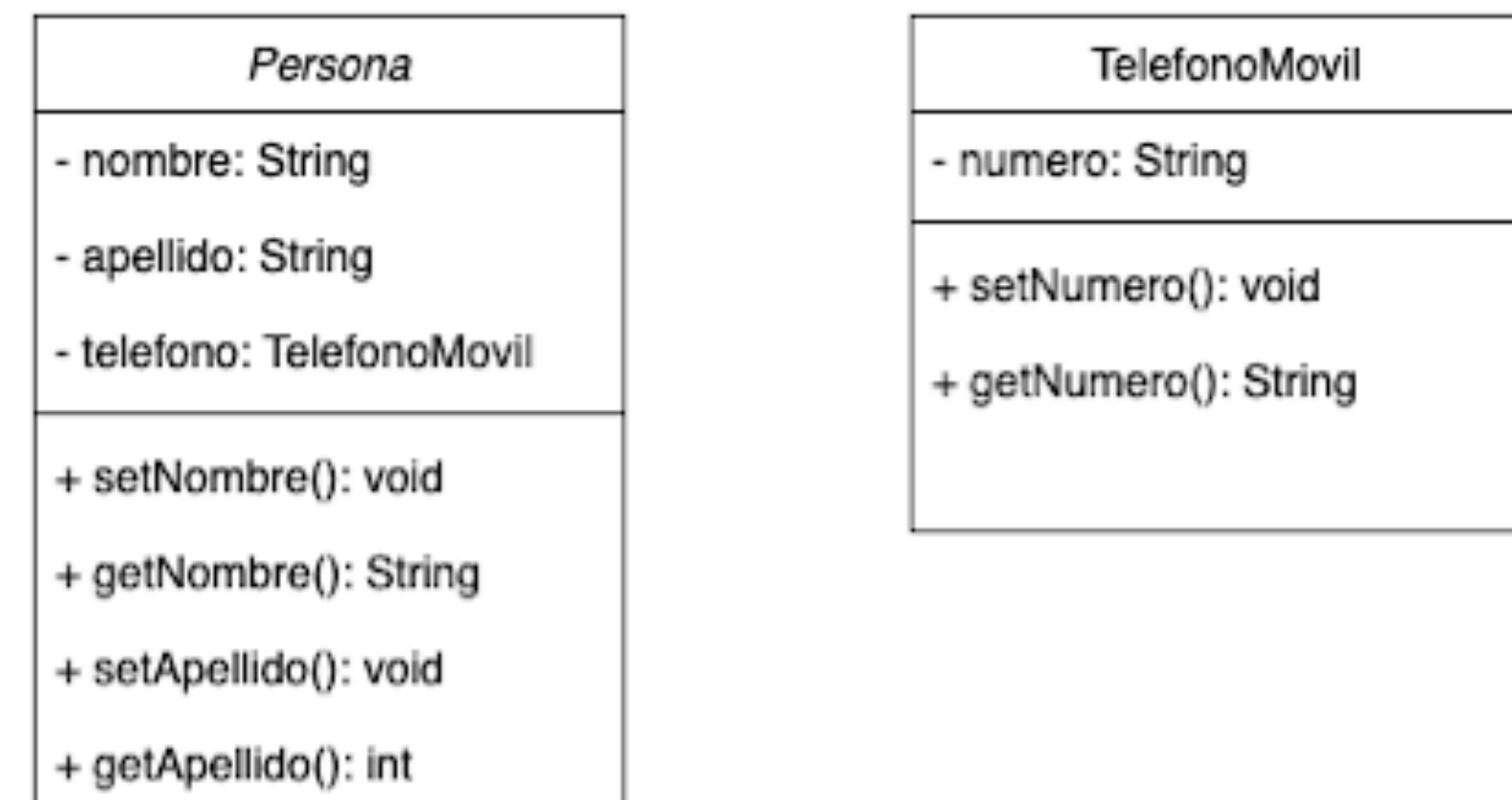
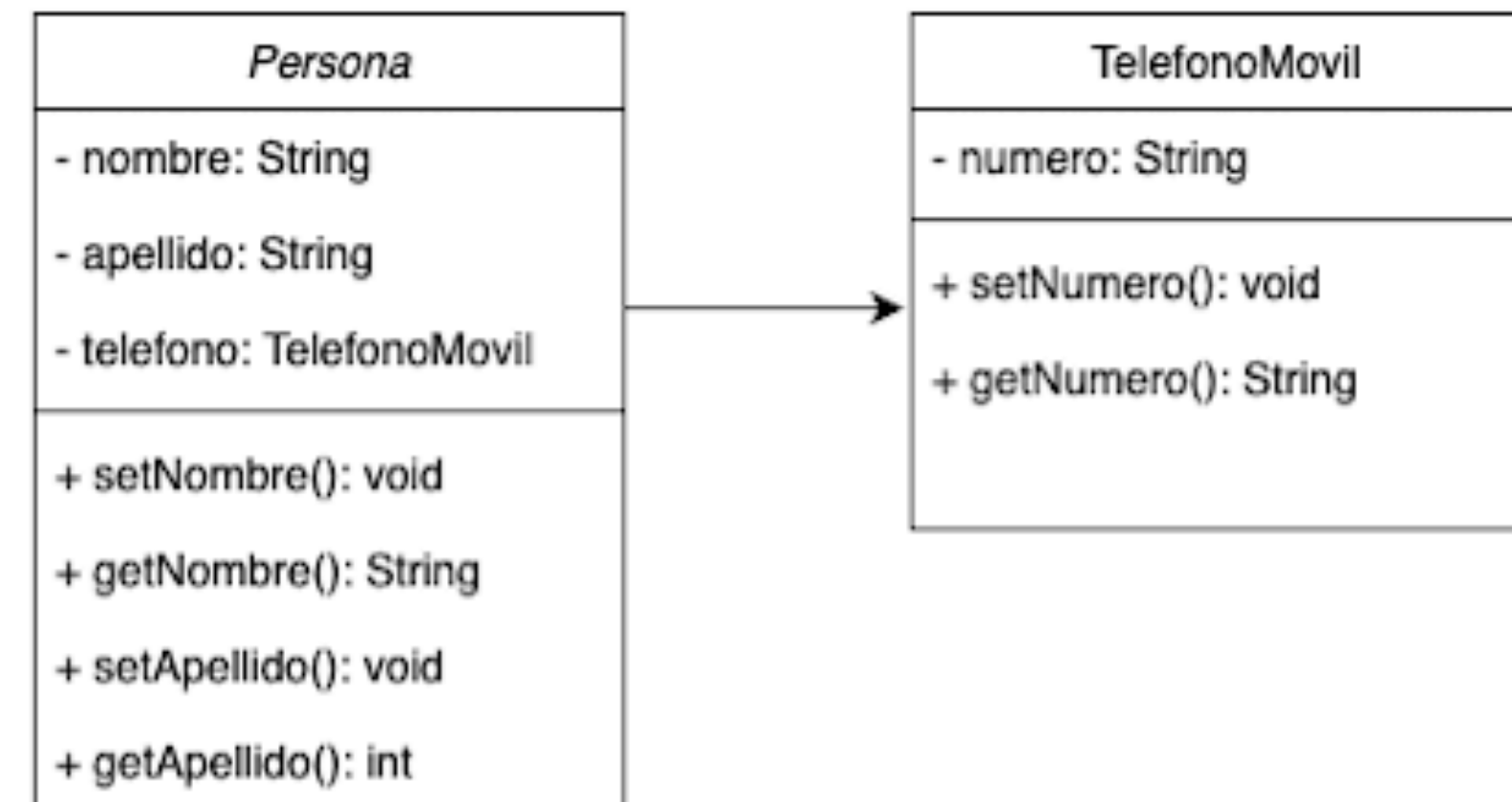
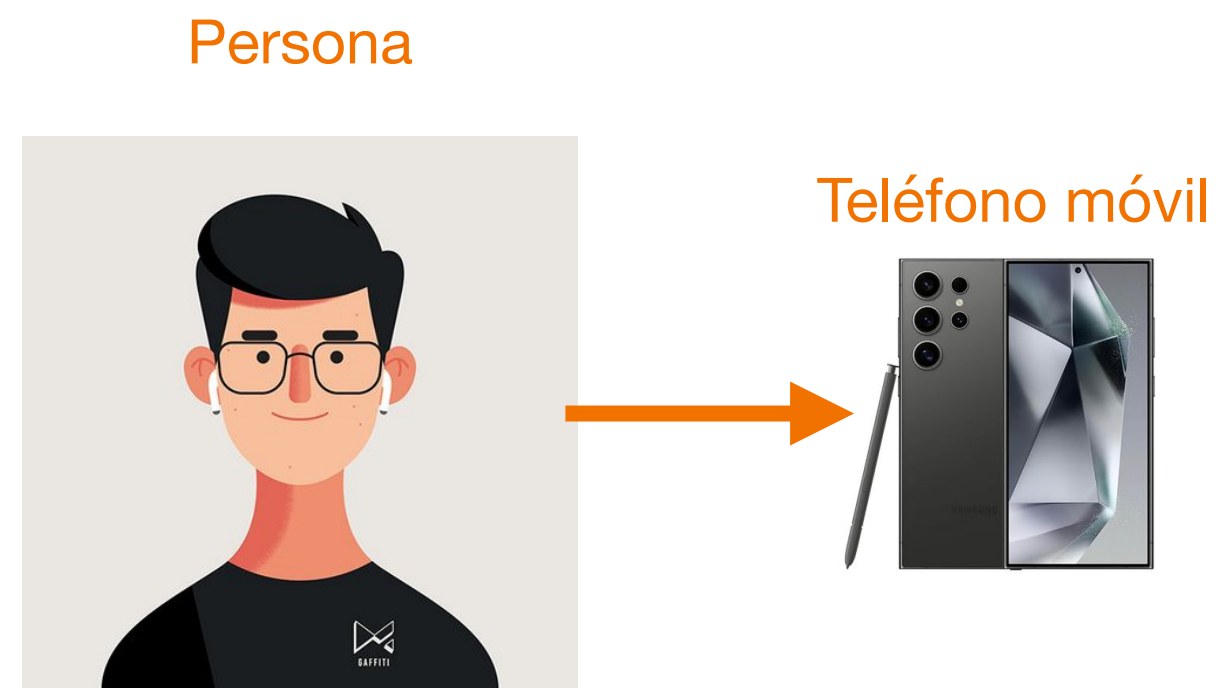


Teléfono móvil

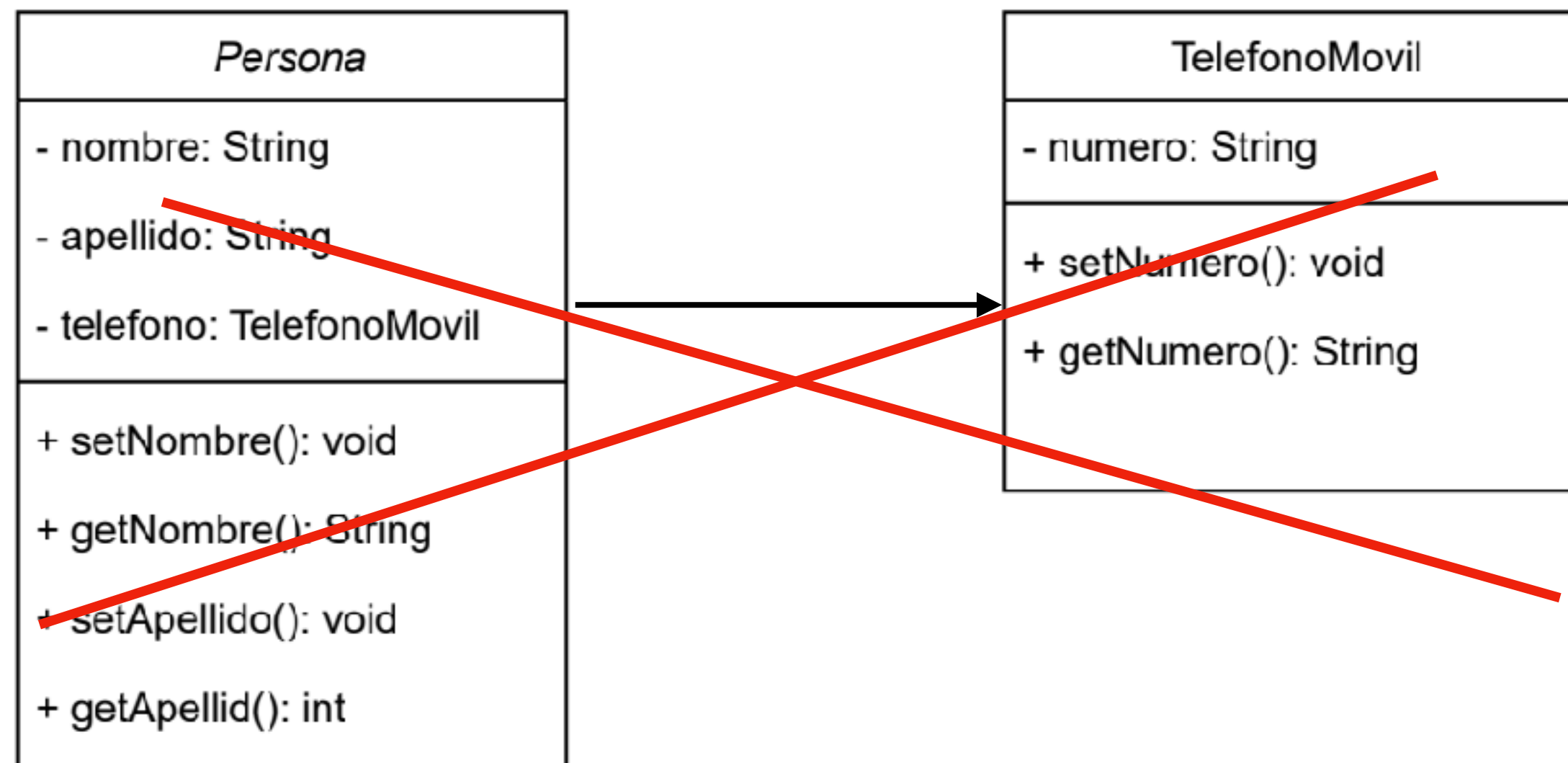
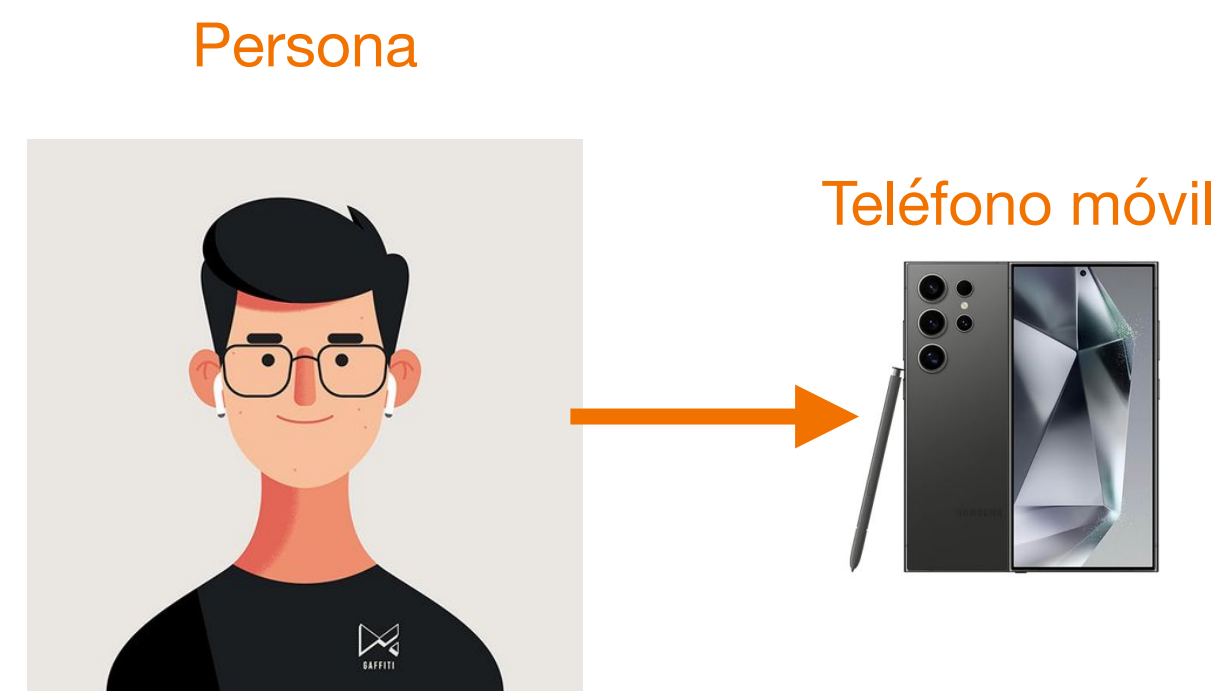


Relación de Asociación

► Representaciones incorrectas en UML



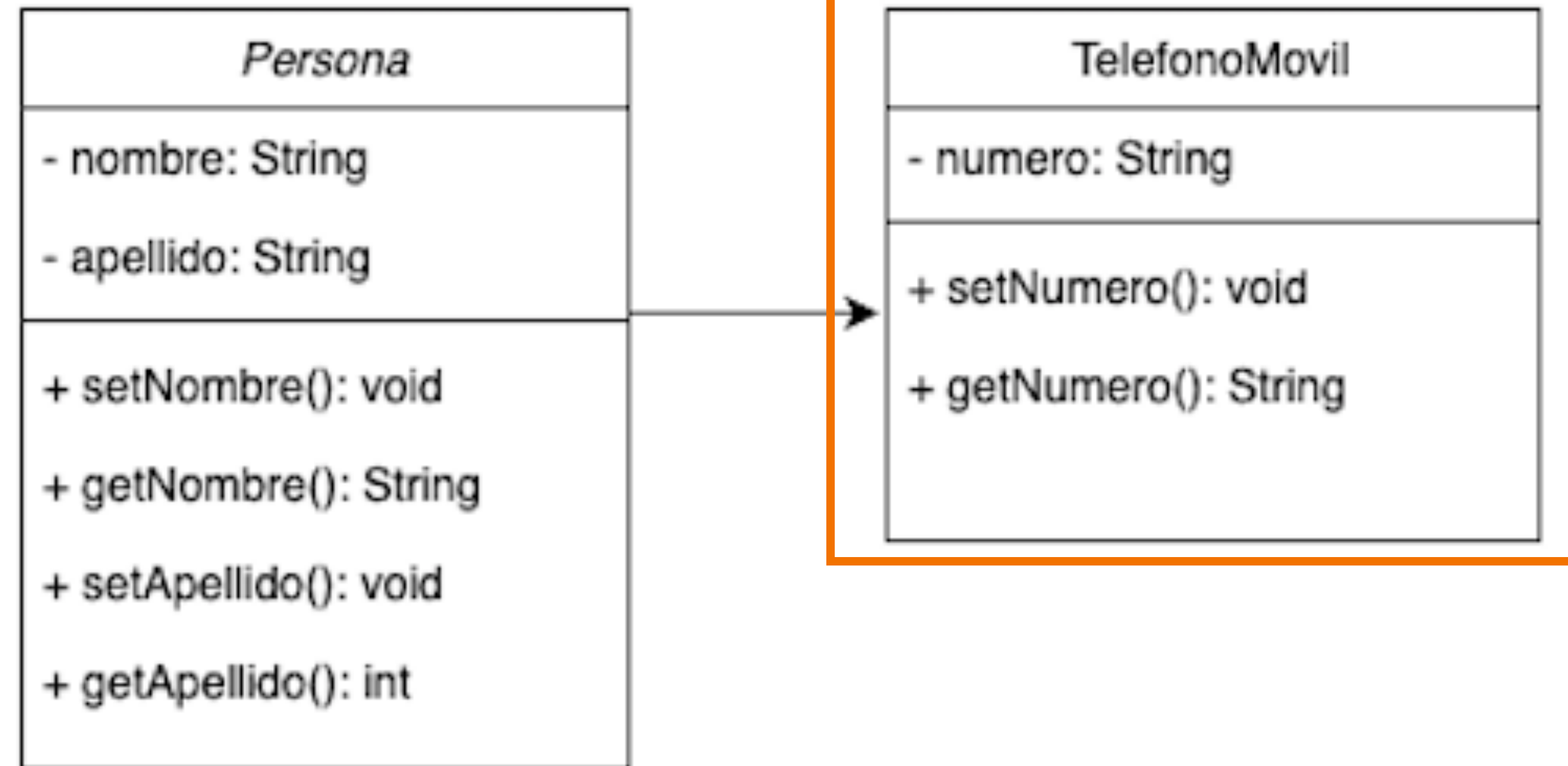
Relación de Asociación



Implementación de Asociación

► Clase TelefonoMovil

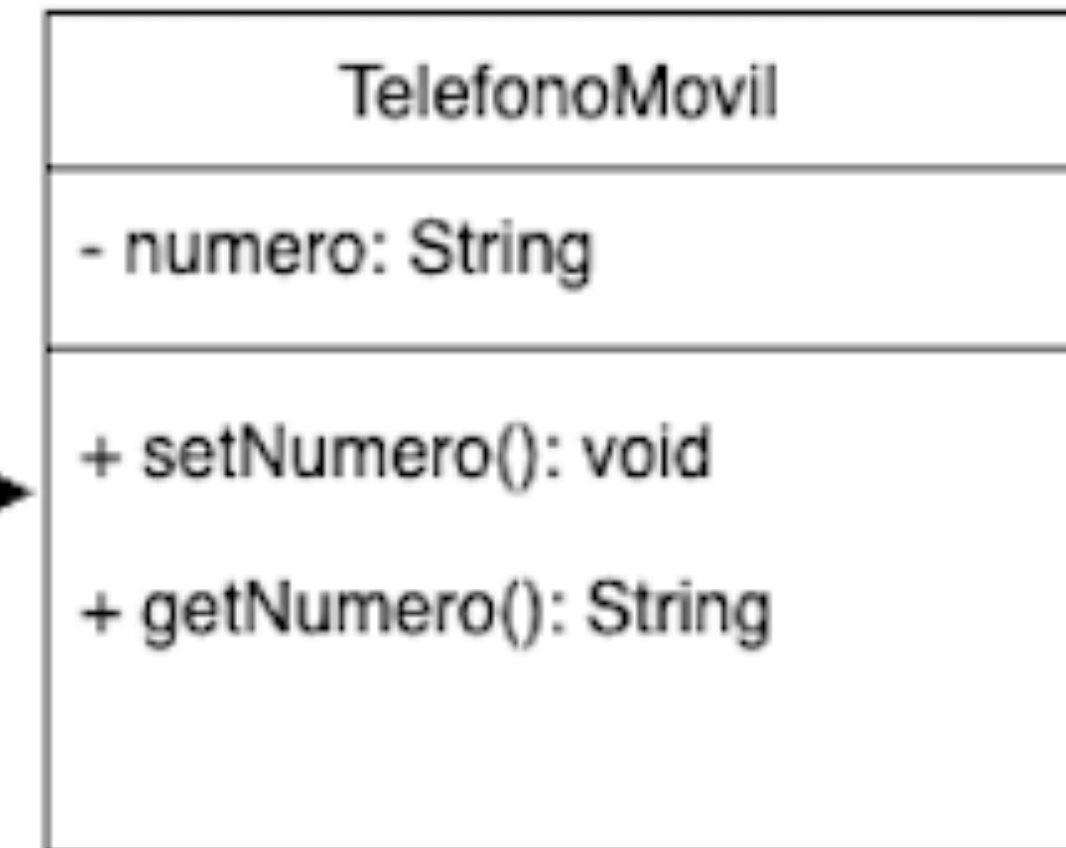
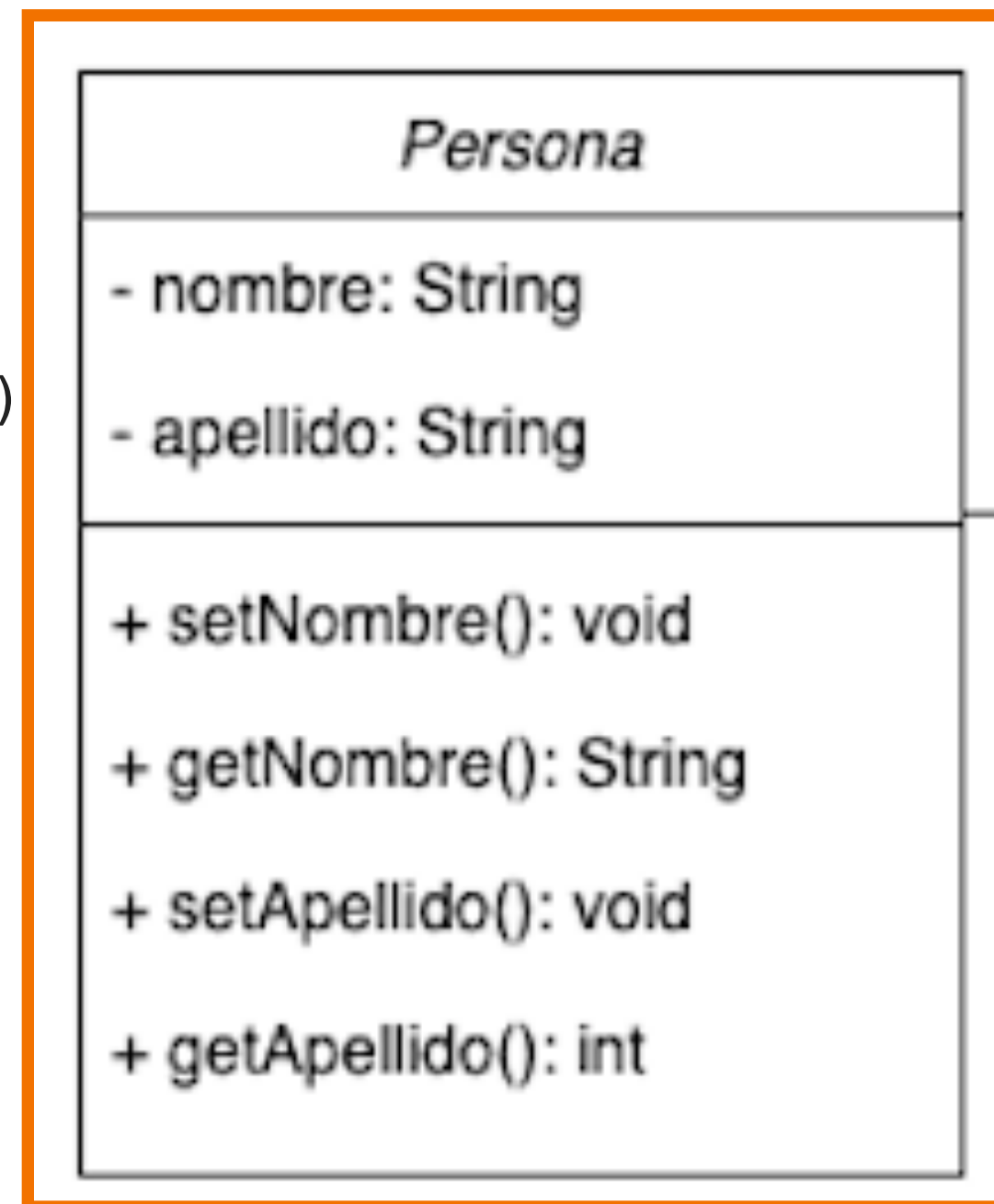
```
public class TelefonoMovil {  
    private String numero;  
  
    public TelefonoMovil(String numero) {  
        this.numero = numero;  
    }  
    public String getNumero() {  
        return numero;  
    }  
    public void setNumero(String numero) {  
        this.numero = numero;  
    }  
    public String toString() {  
        return "Movil: " + numero;  
    }  
}
```



Implementación de Asociación

► Clase Persona

```
public class Persona {  
    private String nombre;  
    private String apellido;  
    private TelefonoMovil telefonoMovil;  
  
    public Persona(String nombre, String apellido)  
        this.nombre = nombre;  
        this.apellido = apellido;  
    }  
    public String getNombre() {  
        return nombre;  
    }  
    public void setNombre(String nombre) {  
        this.nombre = nombre;  
    }  
    public String getApellido() {  
        return apellido;  
    }  
    public void setApellido(String apellido) {  
        this.apellido = apellido;  
    }  
    public String toString() {  
        return nombre + " " + apellido + ", tu telefono es "+telefonoMovil;  
    }  
    public TelefonoMovil getTelefonoMovil() {  
        return telefonoMovil;  
    }  
    public void setTelefonoMovil(TelefonoMovil telefonoMovil) {  
        this.telefonoMovil = telefonoMovil;  
    }  
}
```



Implementación de Asociación

- Clase prueba.

```
public class AsociacionUnaria {  
    public static void main(String[] args) {  
        Persona persona= new Persona("Jose", "Mendez");  
        TelefonoMovil telefonoMovil = new TelefonoMovil("6645123434");  
        persona.setTelefonoMovil(telefonoMovil);  
  
        System.out.println(persona.toString());  
    }  
}
```

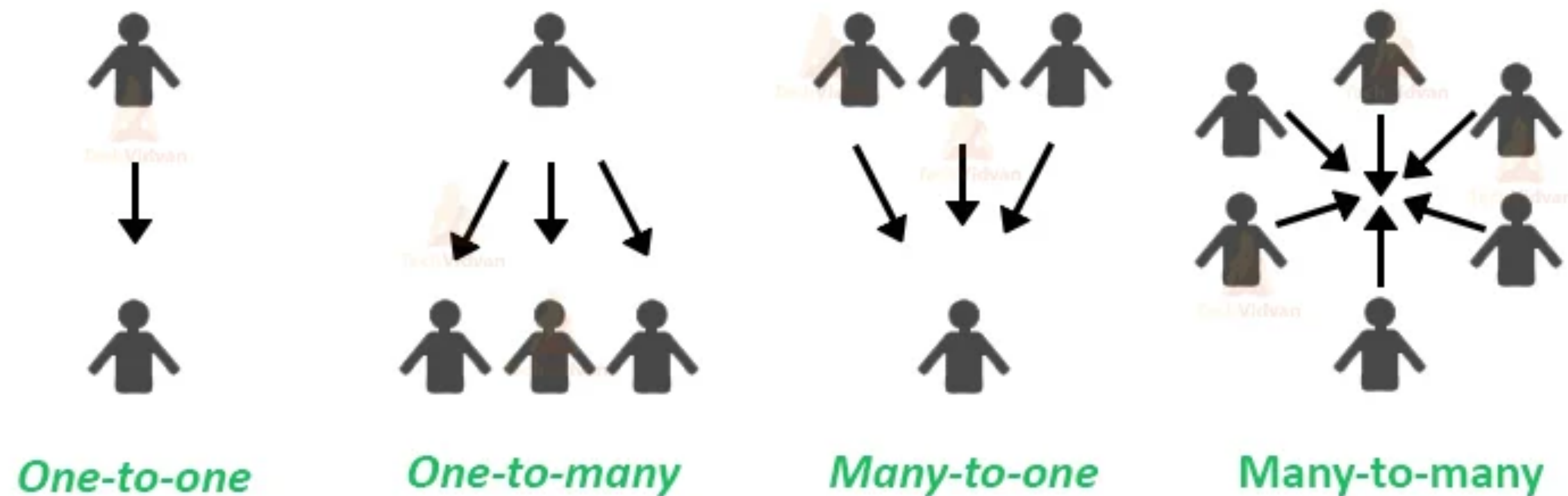
Salida del código

```
Jose Mendez, tu telefono es Movil: 6645123434
```

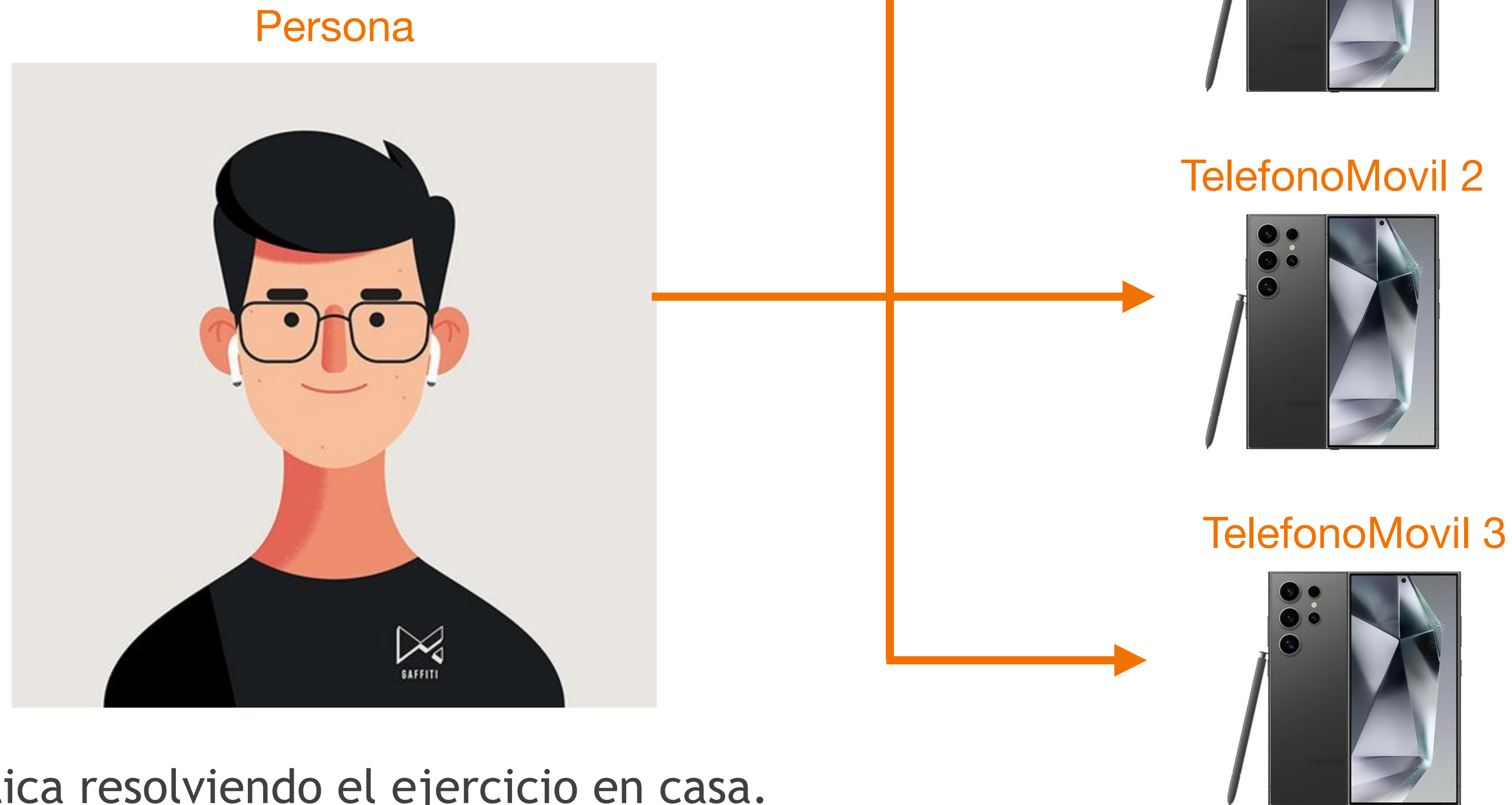

Multiplicidad en la Asociación

- Define cuántos objetos se relacionan: **uno a uno**, **uno a muchos**, **muchos a uno** y **muchos a muchos** entre múltiples clases.

Association in Java



Implementación de Asociación



- Practica resolviendo el ejercicio en casa.
- Una asociación está representada por una línea sólida entre dos objetos de clase que describen la relación.

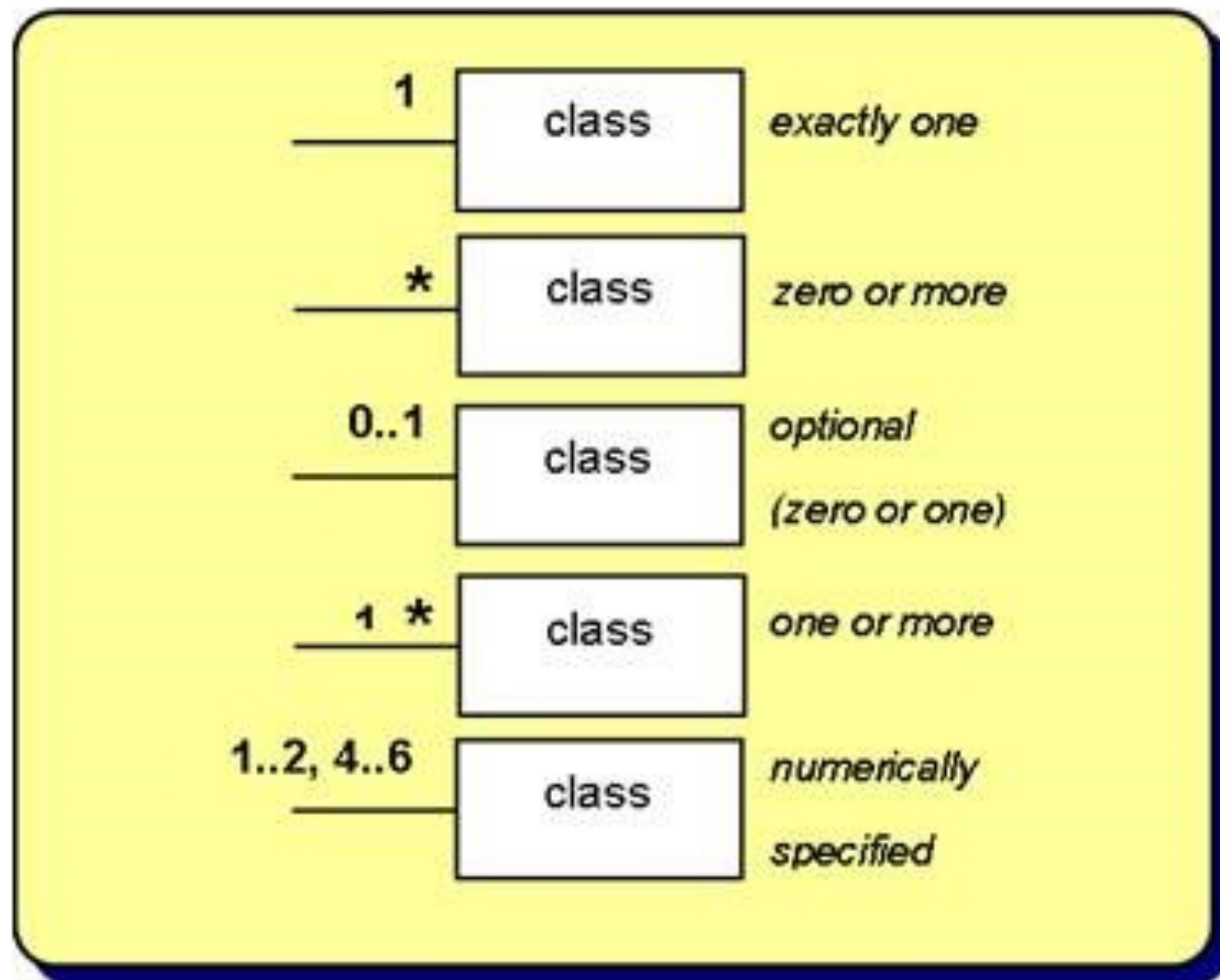
Multiplicidad

- ▶ Ejemplos de cada relación manejados por la asociación:
 - ▶ Un **profesor** solo se puede asignar a trabajar en un **departamento**, relación **uno a uno**
 - ▶ Un **profesor** se puede asignar para trabajar a varios **departamentos**, relación **uno a muchos**
 - ▶ Muchos **profesores** se pueden asignar a un **departamento**, relación **muchos a uno**.
 - ▶ Muchos **profesores** se pueden asignar a trabajar a muchos **departamentos**, relación **muchos a muchos**.



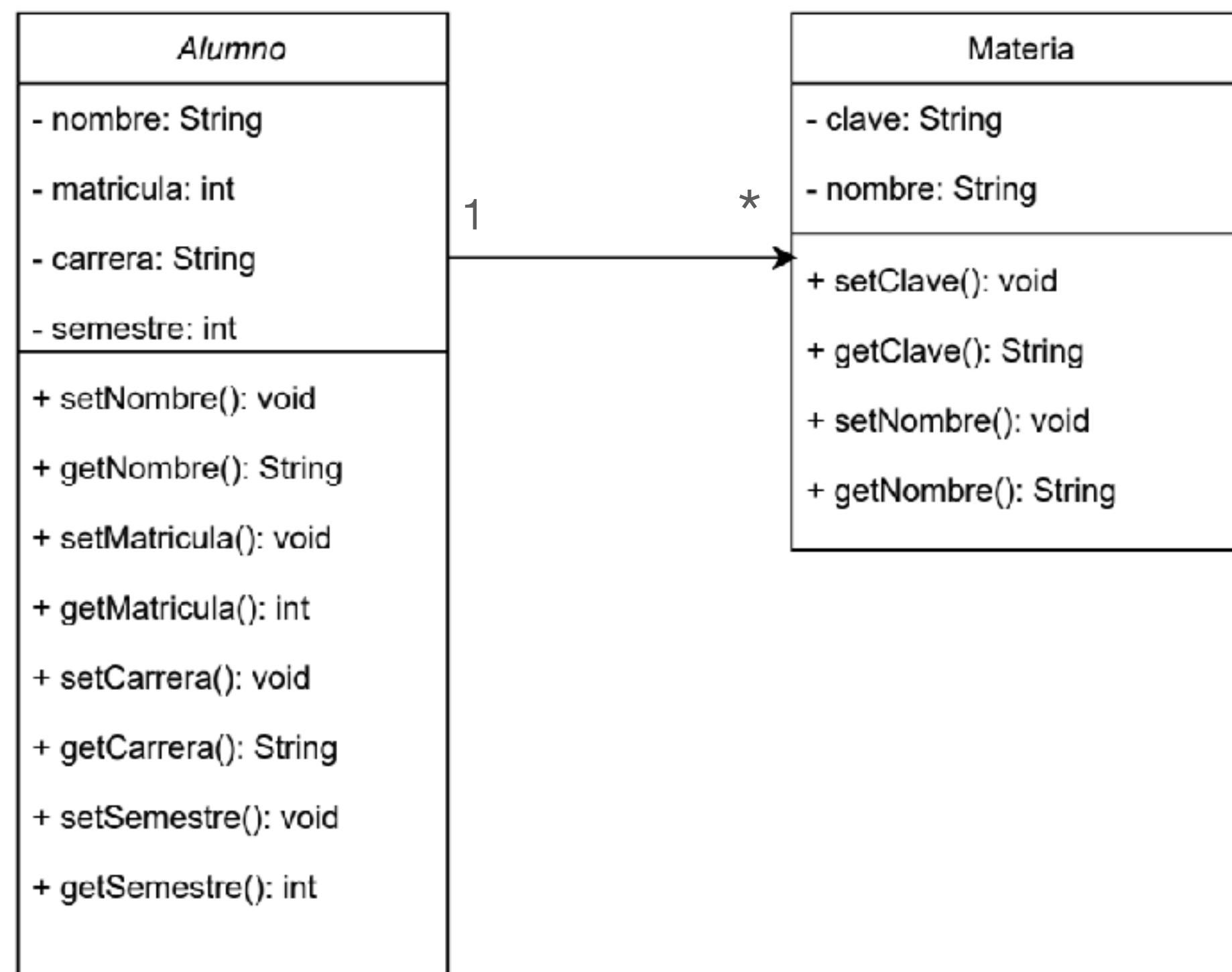
Multiplicidad

- Representación de multiplicidad en UML



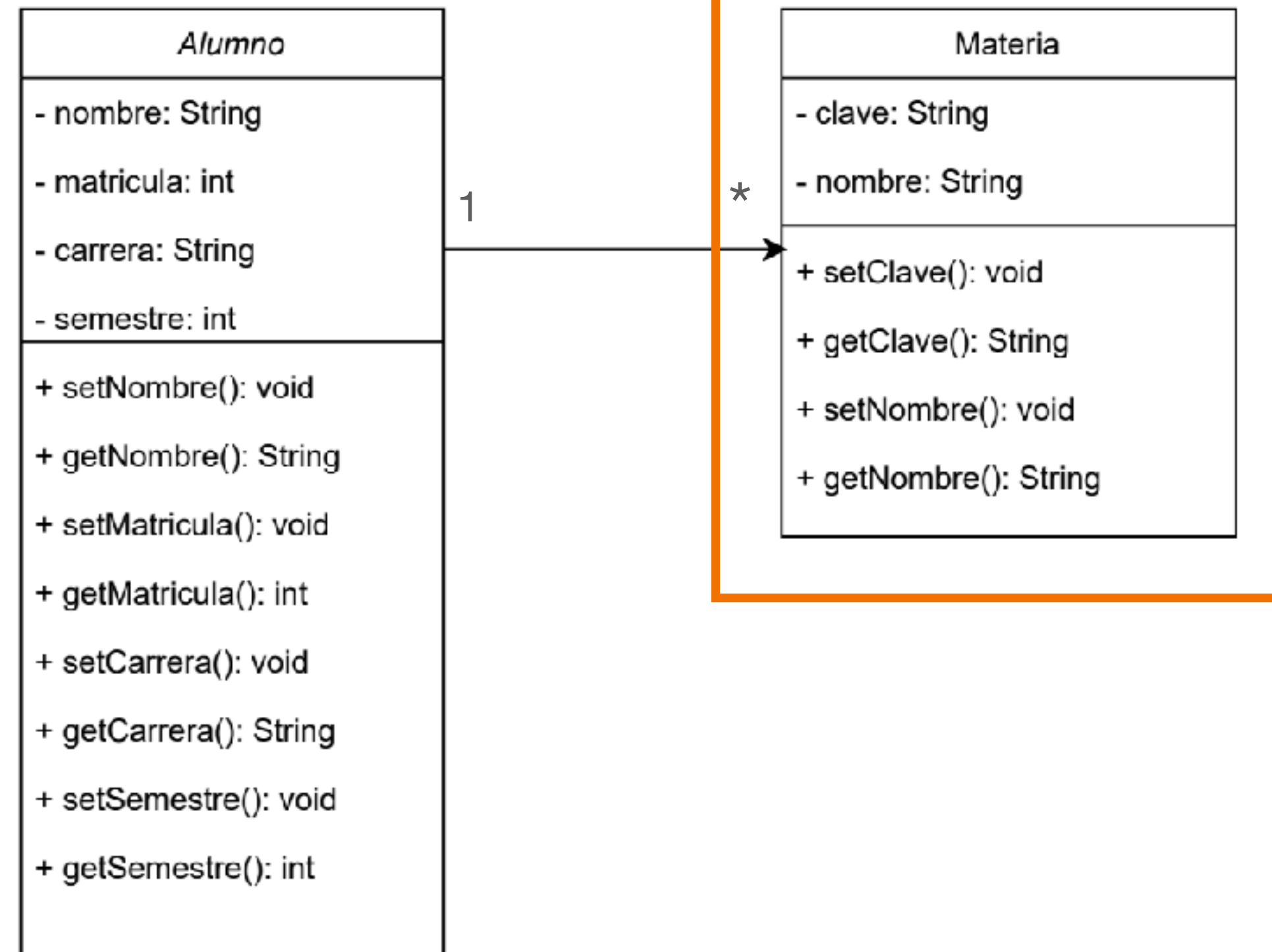
Asociación con Multiplicidad

- Una asociación representa una relación entre dos clases donde una clase está conectada a otra clase de alguna manera



Asociación con Multiplicidad

```
public class Materia {  
    private int clave;  
    private String nombre;  
  
    public void setClave(int clave){  
        this.clave=clave;  
    }  
  
    public void setNombre(String nombre){  
        this.nombre=nombre;  
    }  
  
    public int getClave(){  
        return clave;  
    }  
  
    public String getNombre(){  
        return nombre;  
    }  
}
```

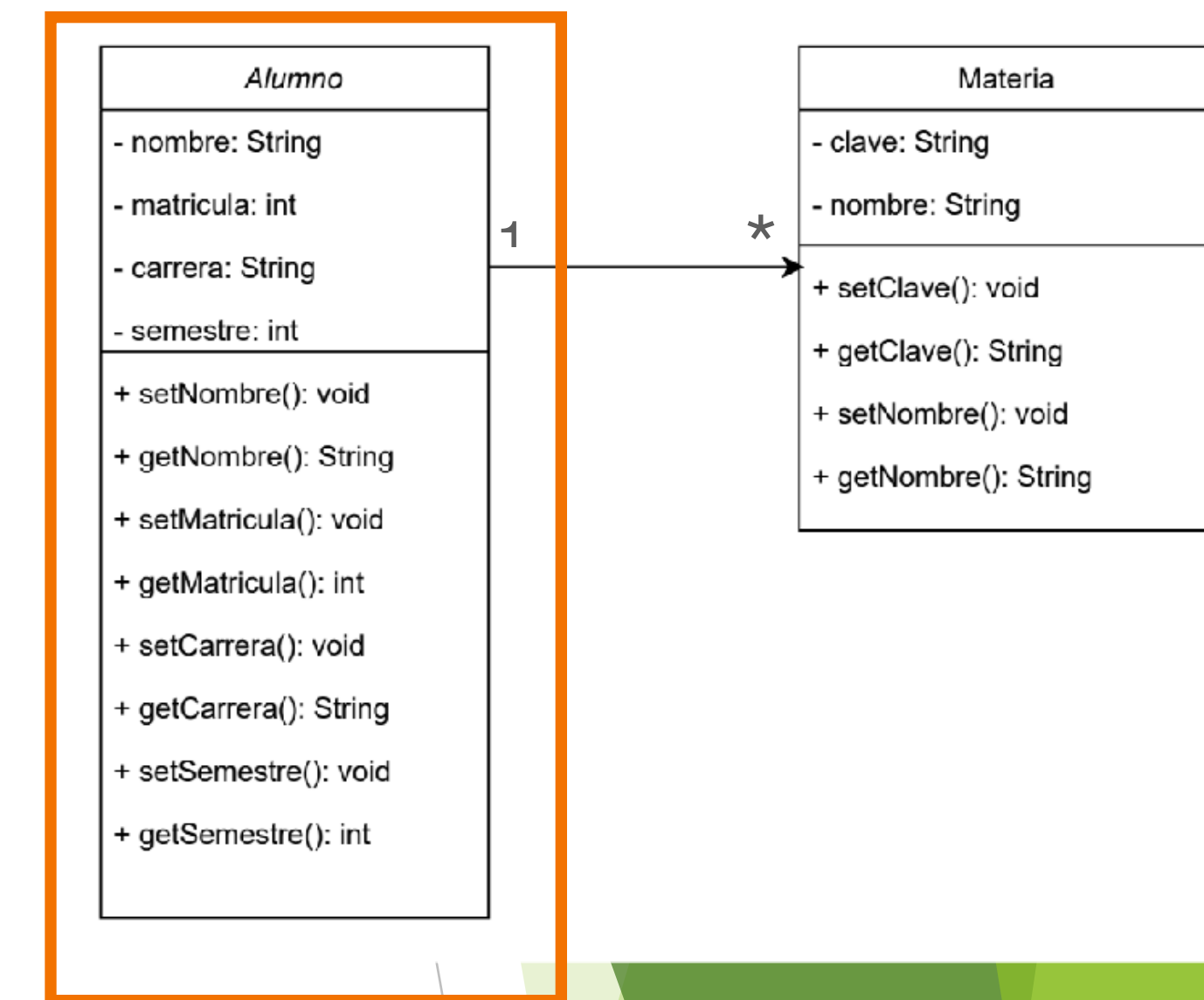


Asociación con Multiplicidad

```
public class Alumno {
    private int matricula;
    private String nombre;
    private String carrera;
    private int semestre;
    private Materia[] listaCursos;
    private int maxMaterias;
    private int contador;

    public Alumno(int matricula, String
nombre, int maxMaterias){
        this.matricula = matricula;
        this.nombre = nombre;
        this.maxMaterias = maxMaterias;
        this.listaCursos = new
Materia[maxMaterias];
        this.contador = 0;
    }
    public void setMatricula(int
matricula){
        this.matricula = matricula;
    }
    public void setNombre(String
nombre){
        this.nombre = nombre;
    }
}
```

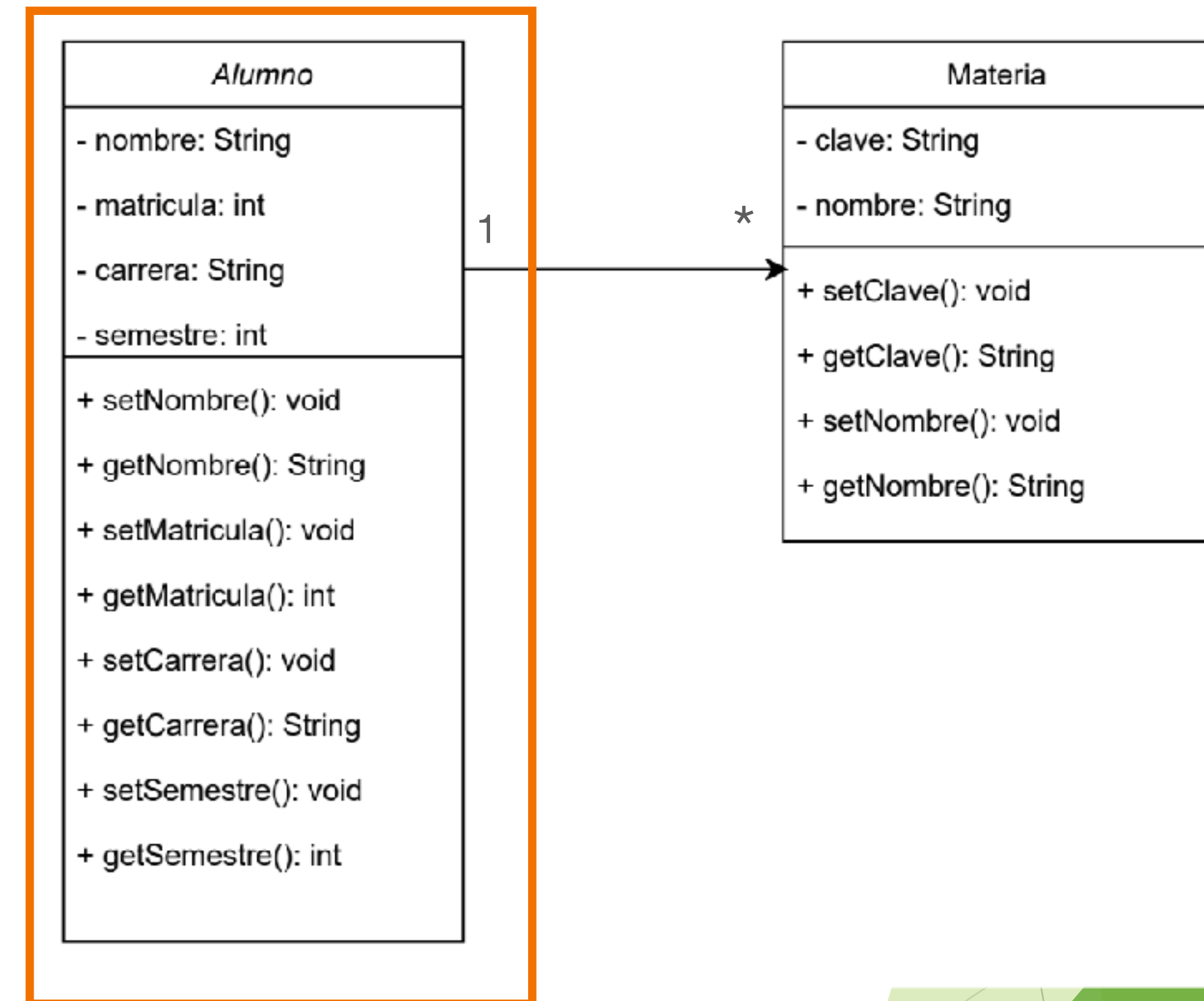
```
    public void setCarrera(String
carrera){
        this.carrera = carrera;
    }
    public void setSemestre(int
semestre){
        this.semestre = semestre;
    }
    public int getMatricula(){
        return matricula;
    }
    public String getNombre(){
        return nombre;
    }
    public String getCarrera(){
        return carrera;
    }
    public int getSemestre(){
        return semestre;
    }
}
```



Asociación con Multiplicidad

```
//metodo para asociar un curso con el Alumno
public void agregarCurso(Materia materia){
    if(contador < maxMaterias)
    {
        listaCursos[contador] = materia;
        contador ++;
    }
    else {
        System.out.println("Maximo de cursos
alcanzados");
    }
}

public void imprimeCursos() {
    System.out.println("Cursos de " + nombre + ":");
    for (int i = 0; i < contador; i++) {
        Materia materia = listaCursos[i];
        System.out.println("Clave: " +
materia.getClave() + ", Nombre: " +
materia.getNombre());
    }
}
}
```



Asociación con Multiplicidad

```
public class EjemploAsociacion{
    public static void main(String [] args)
    {
        Alumno alumno1 = new Alumno(212156,"Jose Lopez
Lopez",2);
        alumno1.setCarrera("Ingeniero en Computacion");
        alumno1.setSemestre(4);

        Materia materia1 = new Materia();
        materia1.setClave(36282);
        materia1.setNombre("Programacion Orientada a
Objetos");

        Materia materia2 = new Materia();
        materia2.setClave(36280);
        materia2.setNombre("Circuitos Electricos");

        alumno1.agregarCurso(materia1);
        alumno1.agregarCurso(materia2);

        alumno1.imprimeCursos();
    }
}
```

Salida del código

```
Cursos de Jose Lopez Lopez:
Clave: 36282, Nombre: Programacion Orientada a Objetos
Clave: 36280, Nombre: Circuitos Electricos
```